

JavaScript Modules, NPM, Vite & Axios

Beginner-to-Advanced Classroom Notes

Coverage: Module systems, CommonJS, ES Modules, export/import, named/default exports, project organization, npm init/install, package.json, package-lock.json, node_modules, npx, Vite/Parcel, development server, HMR, module resolution, production build, minification, tree shaking, code splitting, deployment, Axios with CDN, Axios with NPM, Axios vs Fetch, and Axios interceptors.

Teaching principle: introduce the problem first, then introduce the tool that solves it.

1. Why Do We Need Modules?

A module is a separate unit of code that can expose selected functionality to other files. Modules help divide a large application into smaller, focused files.

```
// Without modules
function add(a, b) {}
function subtract(a, b) {}
function getUsers() {}
function formatDate() {}
function validateForm() {}
```

A better structure is:

```
project/
  main.js
  math.js
  api.js
  validation.js
  utils.js
```

- Better organization and separation of responsibility.
- Reusable code.
- Smaller files that are easier to understand.
- Reduced global-scope pollution.
- Clear dependencies between files.
- Easier testing and maintenance.

2. Module System

A module system defines how code is divided into modules and how one module exposes functionality to another module.

```
JavaScript Module Systems
■
  CommonJS
■
  ES Modules (ESM)
```

The two systems students should recognize are **CommonJS** and **ES Modules**.

3. CommonJS

CommonJS is historically associated with Node.js. It uses **require()** for importing and **module.exports** for exporting.

math.js

```
function add(a, b) {
  return a + b;
}

function subtract(a, b) {
  return a - b;
}

module.exports = {
  add,
  subtract
};
```

app.js

```
const { add, subtract } = require("./math");

console.log(add(10, 20));
console.log(subtract(20, 10));
```

Mental model: module.exports → export, require() → import.

4. ES Modules (ESM)

ES Modules are the standardized JavaScript module system. Modern browsers support ESM natively, and modern frontend tools such as Vite use ESM-style syntax extensively.

math.js

```
export function add(a, b) {  
  return a + b;  
}  
  
export function subtract(a, b) {  
  return a - b;  
}
```

app.js

```
import { add, subtract } from "./math.js";  
  
console.log(add(10, 20));  
console.log(subtract(20, 10));
```

Browser modules

```
<script type="module" src="./main.js"></script>
```

type="module" tells the browser to treat the script as an ES module.

5. Named vs Default Exports

Named exports are useful when a module exposes several specific functions, variables, or classes.

```
// math.js  
export function add(a, b) {  
  return a + b;  
}  
  
export function subtract(a, b) {  
  return a - b;  
}  
  
// main.js  
import { add, subtract } from "./math.js";
```

A module can have one default export. A default import does not use curly braces.

```
// user.js  
export default function getUser() {  
  return { name: "Tarun", age: 25 };  
}  
  
// main.js  
import getUser from "./user.js";
```

The default import name can be changed:

```
import fetchUser from "./user.js";
```

Mixed example:

```
// math.js  
export function add(a, b) { return a + b; }  
export function subtract(a, b) { return a - b; }  
export default function multiply(a, b) { return a * b; }  
  
// main.js  
import multiply, { add, subtract } from "./math.js";
```

Named Export	Default Export
Can have multiple per module	One default export per module

Named Export	Default Export
Imported using {}	Imported without {}
Name normally matches	Import name can be chosen

6. Organizing a Project into Modules

```
product-app/
├── index.html
├── js/
│   ├── main.js
│   ├── api.js
│   ├── ui.js
│   └── utils.js
└── package.json
```

api.js

```
export async function getProducts() {
  const response = await fetch(
    "https://dummyjson.com/products"
  );
  return response.json();
}
```

ui.js

```
export function displayProducts(products) {
  const container = document.querySelector(".container");

  products.forEach(product => {
    const card = document.createElement("div");
    card.innerHTML = `
      <h2>${product.title}</h2>
      <p>${product.price}</p>
    `;
    container.append(card);
  });
}
```

main.js

```
import { getProducts } from "./api.js";
import { displayProducts } from "./ui.js";

async function init() {
  const products = await getProducts();
  displayProducts(products.products);
}

init();
```

7. NPM and npm init

NPM (Node Package Manager) is used to install, manage, and run packages/tools used by JavaScript projects.

```
mkdir product-app
cd product-app
npm init
# or
npm init -y
```

This creates **package.json**.

8. package.json

package.json contains project metadata, scripts, and dependency declarations.

```
{
  "name": "product-app",
```

```

    "version": "1.0.0",
    "scripts": {},
    "dependencies": {}
  }

npm install axios

{
  "dependencies": {
    "axios": "^1.13.2"
  }
}

```

9. npm install and node_modules

```

npm install axios
# short form
npm i axios

```

NPM downloads the package and its required dependencies into **node_modules**, and updates package.json and package-lock.json.

```

project/
├── node_modules/
│   └── axios/
├── package.json
└── package-lock.json

```

Normally, node_modules is not committed to Git because it can be recreated from the package manifests.

10. package-lock.json

package-lock.json records the resolved dependency tree and exact package versions used by NPM. It helps make installations reproducible across machines and environments.

```

package.json
→ What packages does the project need?

package-lock.json
→ What exact dependency tree did NPM resolve?

```

For example, your application may depend on Axios, while Axios may have its own dependency tree. The lock file records the resolved versions and relationships needed for a reproducible installation.

Commit package-lock.json: For application projects, generally yes. It should normally be committed to source control.

Do not normally commit node_modules: put it in .gitignore.

npm ci

```
npm ci
```

npm ci means a clean install and is commonly used in CI/CD environments. It is designed around the lockfile and expects package.json and package-lock.json to be in sync.

11. npx

npx is commonly used to execute package-provided command-line tools without requiring a global installation.

```

npx cowsay "Hello"
npx vite

```

npm mainly manages packages. **npx** commonly executes package tools.

12. Why Do We Need a Build Tool?

As an application grows, we may have many modules, dependencies, and assets. We also want a fast development experience and an optimized production build.

- Resolve imports and dependencies.

- Provide a development server.
- Update changes quickly during development.
- Process and optimize assets.
- Create production-ready output.
- Support tree shaking and code splitting.

Examples include **Vite** and **Parcel**.

13. Vite / Parcel

Vite is a modern frontend development and build tool. Parcel is another tool that provides a development server and build pipeline.

```
npm create vite@latest product-app -- --template vanilla
cd product-app
npm install
npm run dev

product-app/
├── node_modules/
├── public/
├── src/
│   ├── main.js
│   ├── style.css
│   └── index.html
├── package.json
├── package-lock.json
└── vite.config.js
```

14. Development Server and HMR

```
npm run dev
```

HMR = Hot Module Replacement. During development, when a module changes, the tool can update the affected module instead of doing a complete page reload.

Traditional:
Change code → reload whole page → application starts again

HMR:
Change code → detect changed module → update affected module

HMR is not simply 'render only the changed HTML'. It is a development mechanism for updating/replacing changed modules. Framework integrations may also preserve component/application state.

15. Module Import Resolution

There is a difference between a local module import and an NPM package import.

```
// Local module
import { getProducts } from "../api.js";

// NPM package
import axios from "axios";
```

The `./` indicates a relative local path. A bare package name such as **axios** is resolved from installed dependencies by the development/build tooling.

With Vite and NPM, prefer:

```
import axios from "axios";
```

rather than manually importing an internal file from `node_modules`.

16. Production Build

```

npm run dev
# development

npm run build
# production build

npm run preview
# preview production build locally

```

Vite commonly creates a **dist/** directory containing production assets.

```

dist/
  index.html
  assets/
    index-xxxxx.js
    index-xxxxx.css

```

17. Production Optimization

Minification

Minification reduces unnecessary characters such as whitespace and can shorten code representations to reduce file size.

Tree Shaking

Tree shaking removes unused code from the final bundle when the tooling can determine that the code is unused.

```

export function add() {}
export function subtract() {}
export function multiply() {}

import { add } from "./math.js";

// subtract and multiply may be removable
// if they are not used elsewhere.

```

Code Splitting

Code splitting divides application code into smaller chunks so the browser does not necessarily need to download all application code at once.

```

Application
  ■
  ■■■ Home chunk
  ■■■ Admin chunk
  ■■■ Reports chunk

```

18. Deployment

```

Write application
  ↓
npm run build
  ↓
dist/
  ↓
Deploy production files
  ↓
Users access application

```

The exact deployment process depends on the hosting platform, but the key idea is that the production build is generated before deployment.

19. Axios Without NPM — CDN

Axios can be loaded directly from a CDN. This does not require installing Axios into the project with NPM.

```

<!DOCTYPE html>
<html>
<body>

```

```

<div class="container"></div>

<script src="https://cdn.jsdelivr.net/npm/axios@1.13.2/dist/axios.min.js"></script>

<script>
async function fetchData() {
  try {
    const response =
      await axios.get("https://dummyjson.com/products");

    console.log(response);
  } catch (error) {
    console.error(error);
  }
}

fetchData();
</script>
</body>
</html>

```

The CDN script makes Axios available to the page. There is no NPM import in this approach.

20. Axios With NPM

```

npm install axios

import axios from "axios";

async function fetchData() {
  try {
    const response =
      await axios.get("https://dummyjson.com/products");

    console.log(response.data);
  } catch (error) {
    console.error(error);
  }
}

fetchData();

```

With Vite, this is the preferred approach for an NPM-based project.

21. Axios vs Fetch

```

// Fetch
const response =
  await fetch("https://dummyjson.com/products");

const data =
  await response.json();

```

Fetch is a browser Web API. It returns a Response object, and JSON parsing is a separate asynchronous step.

```

// Axios
const response =
  await axios.get("https://dummyjson.com/products");

console.log(response.data);

```

Axios is an external HTTP client library. In the usual JSON workflow, the parsed response body is available through `response.data`.

Fetch	Axios
Built into modern browsers	External library
No installation needed	Install with NPM or use CDN
Returns a Response object	Returns an Axios response object
Usually call <code>response.json()</code>	JSON body commonly available as <code>response.data</code>

Fetch	Axios
404/500 do not automatically reject	Non-2xx responses reject by default
No Axios-style interceptors	Provides request/response interceptors

Fetch HTTP Error Handling

Fetch rejects for network-level failures, but a normal HTTP error such as 404 or 500 does not automatically reject the promise. Check `response.ok` or the status yourself.

```
try {
  const response =
    await fetch("https://dummyjson.com/products");

  if (!response.ok) {
    throw new Error(
      `HTTP Error: ${response.status}`
    );
  }

  const data = await response.json();
} catch (error) {
  console.error(error);
}
```

22. Axios Interceptors

Interceptors allow code to run before a request is sent or after a response is received. They are useful for logging, authentication headers, and centralized error handling.

```
axios.interceptors.request.use(config => {
  console.log("Request sent");
  return config;
});

axios.interceptors.request.use(config => {
  config.headers.Authorization = "Bearer TOKEN";
  return config;
});
```

23. Complete Modern JavaScript Project Flow

```
Problem: one large JavaScript file
↓
Why modules?
↓
Module System
↓
CommonJS + ES Modules
↓
export / import
↓
Named / Default exports
↓
Organize project into modules
↓
NPM
↓
npm init
↓
package.json
↓
npm install
↓
node_modules
↓
package-lock.json
↓
npx
↓
```

Need better frontend development workflow
↓
Vite / Parcel
↓
Development server
↓
Module resolution
↓
HMR
↓
Production build
↓
Minification / Tree shaking / Code splitting
↓
Deployment
↓
Axios
↓
CDN approach
↓
NPM approach
↓
Axios vs Fetch
↓
Interceptors

24. Quick Revision

- What problem does a module system solve?
- What is the difference between CommonJS and ES Modules?
- What is the difference between export and import?
- What is the difference between named and default exports?
- Why is type="module" used in browser-based ES Modules?
- What is NPM?
- What does npm init do?
- What is package.json?
- What is node_modules?
- What is package-lock.json and why should it generally be committed?
- What is the difference between npm and npx?
- Why do frontend projects use Vite or Parcel?
- What is a development server?
- What is HMR?
- What is module resolution?
- What happens during a production build?
- What are minification, tree shaking, and code splitting?
- What is the difference between Axios via CDN and Axios via NPM?
- What is the difference between Axios and Fetch?
- Why do we check response.ok with Fetch?
- What are Axios interceptors?

Final takeaway: Modules organize JavaScript, NPM manages dependencies, package.json describes the project, package-lock.json records the resolved dependency tree, node_modules contains installed packages, Vite/Parcel provide development and build tooling, and Axios provides a higher-level HTTP client option.